

PUF-Based Authentication for UAV Systems: A Comprehensive Guide

Executive Summary

Physical Unclonable Functions (PUFs) provide lightweight, hardware-based authentication for resource-constrained Unmanned Aerial Vehicles (UAVs) by exploiting inherent manufacturing variations to create unique device fingerprints. PUF-based protocols achieve 5-92% computational efficiency improvements, 9-68% lower communication overhead, and 22-48% reduced storage requirements compared to traditional cryptographic methods. This guide covers PUF fundamentals, challenge-response pairs, implementation architectures, lightweight cryptographic primitives, and practical deployment strategies for UAV authentication systems.[1][2][3]

What Are Physical Unclonable Functions?

Core Concept

A Physical Unclonable Function is a hardware security primitive that exploits uncontrollable manufacturing process variations in integrated circuits to generate unique, device-specific responses. These microscopic variations—differences in wire delays, threshold voltages, or oscillator frequencies—create a unique "fingerprint" for each chip, similar to human biometrics.[4][5][6]

The fundamental property of a PUF is the challenge-response relationship, formally modeled as:

$$R = \text{PUF}(C, \Phi)$$

where C is the digital challenge input, Φ represents the random physical disorder from fabrication, and R is the corresponding

response. The mapping from challenge to response cannot be duplicated without the specific physical instance, making PUFs inherently unclonable.[4]

Why PUFs for UAVs?

UAVs face unique security challenges: limited computational power, constrained battery life, operation in hostile environments, and vulnerability to physical capture attacks. Traditional Public-Key Infrastructure (PKI) solutions require prohibitive computation and power consumption for lightweight drones. PUFs eliminate the need to store cryptographic keys in non-volatile memory, which is vulnerable to reverse engineering and side-channel attacks.[2][3][7][8][9][1]

Key advantages for UAV systems include:

- **No key storage required:** Keys are generated on-demand from hardware[7][10]
- **Lightweight operations:** Uses simple XOR and hash functions instead of complex asymmetric cryptography[11][2]
- **Physical attack resistance:** PUF responses change if tampered with[12][13][1]
- **Low power consumption:** Minimal energy overhead suitable for battery-powered drones[3][14]

Challenge-Response Pairs (CRPs)

Understanding CRPs

Challenge-Response Pairs form the foundation of PUF-based authentication. A CRP consists of an input challenge C_i and its corresponding output response R_i :

$$\text{CRP}_i = (C_i, R_i) \text{ where } R_i = \text{PUF}(C_i)$$

The challenge is typically a digital bit-string (e.g., 64-bit, 128-bit) that configures the PUF circuit, and the response is the resulting output bit-string (e.g., 128-bit, 256-bit).[10][15][16]

CRP Properties

For secure authentication, CRPs must satisfy critical properties:[16]

Uniqueness: Different PUF devices produce different responses for the same challenge. The inter-chip Hamming distance quantifies this:

$$\text{Uniqueness} = \frac{2}{k(k-1)} \sum_{i=1}^{k-1} \sum_{j=i+1}^k \frac{HD(R_i, R_j)}{n} \times 100\%$$

where k is the number of devices, n is the response bit-length, and HD is the Hamming distance. The ideal uniqueness value is 50%, indicating that approximately half the response bits differ between devices.[17][18][19][20]

Reliability: The same PUF device produces consistent responses for the same challenge under varying environmental conditions. The intra-chip Hamming distance measures this:

$$\text{Reliability} = \frac{1}{k} \sum_{i=1}^k \frac{HD(R_i, R'_i)}{n} \times 100\%$$

where R_i is the reference response and R'_i is the response under different conditions. The ideal reliability value approaches 0% (Bit Error Rate close to 0), though practical PUFs exhibit 2-5% BER requiring error correction.[21][22][18][20][17]

Uniformity: Response bits should be balanced with approximately equal numbers of 0s and 1s:

$$\text{Uniformity} = \frac{1}{k} \sum_{i=1}^k \left(\frac{1}{n} \sum_{j=1}^n r_{i,j} \right) \times 100\%$$

where $r_{i,j}$ is the j -th bit of the i -th response. The ideal uniformity is 50%.[23][24][25][20][21]

Unpredictability: Knowing some CRPs should not enable prediction of other CRPs. This property distinguishes strong PUFs (exponentially many CRPs) from weak PUFs (limited CRPs).[6][16][4]

CRP Space Classification

Weak PUFs generate limited CRPs (typically 2^7 to 2^{10}), suitable only for key generation. Examples include SRAM PUFs and Butterfly PUFs. [26][27][6]

Strong PUFs produce exponentially large CRP spaces (2^{64} or more), enabling authentication protocols. Examples include Arbiter PUFs and Ring Oscillator PUFs. [8][6][16]

PUF Types and Architectures

Arbiter PUF (Delay-Based)

Arbiter PUFs exploit random propagation delay variations in symmetric paths. An n -bit challenge configures n multiplexer stages, creating two parallel delay paths. Signals race through these paths, and an arbiter (SR latch or D flip-flop) determines which arrives first, generating a 1-bit response. [28][9][29][8]

Working principle: The challenge bits select which inputs connect to outputs at each multiplexer stage. Due to manufacturing variations, one path accumulates slightly more delay than the other. The arbiter compares arrival times and outputs 1 or 0 based on the winner. [9][8]

Advantages:

- Exponentially large CRP space: 2^n possible challenges from n stages
- Simple hardware: only multiplexers, wires, and one arbiter [8]
- Strong PUF suitable for authentication protocols

Disadvantages:

- Vulnerable to machine learning modeling attacks with sufficient CRPs [30][31]
- Sensitive to temperature and voltage variations [32][33]
- Requires careful path balancing in FPGA implementations [29]

Practical implementation: For UAV authentication, 64-stage to 128-stage Arbiter PUFs provide adequate CRP space while maintaining hardware efficiency. Recent improvements use XOR Arbiter PUFs

(multiple arbiter chains with XORed outputs) to resist modeling attacks.[34][31][7][30]

Ring Oscillator PUF (RO-PUF)

RO-PUFs compare oscillation frequencies of identically-designed ring oscillators to generate responses. Each ring oscillator consists of an odd number of inverters connected in a feedback loop, creating continuous oscillation.[35][5][36][37][28]

Working principle: Despite identical designs, manufacturing variations cause each oscillator to run at a slightly different frequency. The challenge selects pairs of oscillators to compare. Counters measure oscillation frequencies over a fixed time period, and a comparator determines which oscillates faster, producing one response bit.[37][28]

For N ring oscillators, the number of possible pairwise comparisons is:

$$\text{CRPs} = \binom{N}{2} = \frac{N(N-1)}{2}$$

Advantages:

- Naturally reliable due to relative frequency comparison (less sensitive to environmental changes)[36][38]
- Easy FPGA implementation without complex routing constraints[5][39]
- More resistant to modeling attacks than Arbiter PUFs[40]

Disadvantages:

- Limited CRP space compared to Arbiter PUFs (quadratic vs. exponential)[36][16]
- Higher hardware overhead: requires multiple oscillators and counters[14][36]

Enhanced designs: Configurable RO-PUFs increase CRP space by using the threshold frequency parameter as an additional challenge input. Two methods enhance CRPs:[16]

1. **Frequency threshold selection:** Different threshold values create different groupings of oscillators
2. **RO subset selection:** Choose different subsets of oscillators to compare

With these techniques, CRP counts can reach 10^{10} to 10^{50} for practical implementations.[16]

SRAM PUF (Memory-Based)

SRAM PUFs exploit the random power-up state of Static RAM cells. Each SRAM cell consists of cross-coupled inverters that prefer to initialize to either 0 or 1 based on microscopic transistor mismatches. [28][26][37]

Working principle: When power is applied, each SRAM cell settles into its preferred state (0 or 1) determined by transistor threshold voltage variations. By reading the power-up values of multiple SRAM cells, a unique bit-string is extracted. The cell addresses serve as implicit challenges.[12][26][37][23]

Advantages:

- Ubiquitous availability: standard SRAM exists in nearly all microcontrollers and FPGAs[6][12]
- Zero hardware overhead if using existing memory[12]
- Simple readout procedure

Disadvantages:

- Weak PUF: limited CRP space (only as many responses as SRAM cells)[27][6]
- Requires power cycling for each response generation (impractical for frequent authentication)[26][37]
- Higher bit error rates (5-15%) requiring robust error correction[41][22][21]

UAV application: SRAM PUFs are suitable for initial device enrollment and key generation, but not for repeated authentication due to power cycling requirements.[42][12]

Butterfly PUF (Memory-Based)

Butterfly PUFs use cross-coupled latches that can be forced into a metastable state and then released to settle based on manufacturing variations. Unlike SRAM PUFs, they don't require power cycling and can be implemented on all FPGAs.[43][30][26]

Working principle: Two cross-coupled latches are first preset to opposite values (forcing metastability), then simultaneously released. Random transistor mismatches determine which latch "wins" and pulls the other.[43][26]

Advantages:

- No power cycling required (evaluable without reset)[43]
- FPGA-friendly: works on devices that don't support uninitialized SRAM readout
- Lower bit error rates than SRAM PUFs

Disadvantages:

- Weak PUF with limited CRP space[30][26]
- Additional hardware compared to reusing existing SRAM

Comparison Summary

PUF Type	CRP Space	Strength	Reliability	Hardware Cost	ML Attack Resistance	Best For
Arbit er	Expo nenti al (2^n)	Stro ng	Moder ate (3-5% BER)	Low	Low (unless XORed)	Authenti cation[8] [9]
Ring Oscil lator	Large (quad ratic)	Stro ng/ Wea k	High (1-3% BER)	Mode rate	High	Authenti cation, Key Gen[5] [36]
SRA M	Small (linea r)	Wea k	Moder ate (5-15% BER)	Zero (reus e)	N/A	Key Generati on[26] [37]
Butt erfly	Small (linea r)	Wea k	Moder ate-High	Low	N/A	Key Generati on[43] [30]

Lightweight Hash Functions for UAVs

Why Lightweight Cryptography?

UAVs demand cryptographic primitives optimized for minimal area, low power consumption, and fast execution on resource-constrained hardware. Traditional hash functions like SHA-256 consume excessive power and silicon area for embedded drone processors.[44] [45][46]

SPONGENT Hash Function Family

SPONGENT is a lightweight hash function family specifically designed for resource-constrained devices. It uses a sponge construction based on PRESENT-type permutations, following the hermetic sponge strategy.[47][48][49][44]

Variants: SPONGENT provides five variants with different output sizes:

- **SPONGENT-88:** 88-bit output (preimage resistance only, ultra-constrained scenarios)
- **SPONGENT-128:** 128-bit output (low collision security requirements)
- **SPONGENT-160:** 160-bit output (SHA-1 compatibility, moderate security)
- **SPONGENT-224:** 224-bit output (SHA-2 compatibility)
- **SPONGENT-256:** 256-bit output (SHA-2/SHA-3 compatibility)[48][47]

Hardware requirements: The smallest ASIC implementations require:

- SPONGENT-88: 738 gate equivalents (GE)
- SPONGENT-128: 1,060 GE
- SPONGENT-160: 1,329 GE
- SPONGENT-224: 1,728 GE
- SPONGENT-256: 2,004 GE[49][47]

For comparison, SHA-256 requires approximately 10,868 GE, making SPONGENT-256 about 5× more efficient in silicon area.[47]

Performance: SPONGENT achieves 52% memory reduction and 37% energy reduction compared to AES-based constructions, while maintaining equivalent security guarantees.[44]

PHOTON Hash Function

PHOTON is another sponge-based lightweight hash function with even lower hardware footprint than SPONGENT in some configurations. It uses an AES-like internal permutation optimized for lightweight implementation.[48]

PRESENT Block Cipher

While primarily a block cipher, PRESENT (64-bit block, 80/128-bit key) is often used in hashing modes for authentication protocols. Its lightweight design (1,570 GE for PRESENT-80) makes it suitable for UAV applications.[44][47]

Practical Selection for UAV PUF Protocols

For PUF-based UAV authentication, the recommended approach combines:

1. **SPONGENT-128** or **SPONGENT-160** for response hashing and message authentication[50][51][1]
2. **XOR operations** for lightweight encryption/masking of PUF responses[52][2][11]
3. **Timestamps/nonces** for replay attack prevention[1][11]

This combination provides strong security with minimal overhead. Recent UAV protocols demonstrate that using only hash functions and XOR operations achieves authentication in under 10 ms on Raspberry Pi 4B hardware (representative of drone computational capability).[1]

Error Correction Mechanisms

Why Error Correction Is Critical

PUF responses contain inherent noise due to environmental variations (temperature, voltage, aging). Typical bit error rates range from 2-5% under normal conditions, increasing to 10-15% under extreme temperature/voltage swings. Without error correction, authentication would fail frequently, rendering PUFs impractical.[22][53][54][21]

Fuzzy Extractors

A fuzzy extractor is a cryptographic primitive that extracts stable, uniform randomness from noisy sources. It consists of two procedures:[53][54][55]

Generation (Enrollment Phase):

- Input: Noisy PUF response w
- Output: Extracted key R and public helper data P

Reproduction (Authentication Phase):

- Input: Noisy response w' and helper data P
- Output: Recovered key R if w and w' are "close" (differ in few bits)

The helper data enables error correction without revealing information about the original response.[54][53]

BCH Codes

Bose-Chaudhuri-Hocquenghem (BCH) codes are widely used for PUF error correction due to efficient syndrome-based decoding.[56][57][53]

BCH(n, k, t) parameters:

- n : Codeword length (e.g., 255 bits)
- k : Message length (e.g., 131 bits)
- t : Error correction capability (e.g., 18 errors)[53]

Encoding: The message (PUF response) is encoded using a generator polynomial to produce a codeword. The parity bits (syndrome) serve as helper data:

$$\text{Helper Data} = w \times H^T$$

where H^T is the parity-check matrix transpose.[58][59]

Decoding: During authentication, the noisy response w' is decoded using the helper data:

$$w = \text{Decode}(w' \oplus \text{Helper Data})$$

Practical example: A BCH(255, 131, 18) code corrects up to 18 bit errors in a 255-bit PUF response, sufficient for typical 2-5% BER. For 128-bit key extraction, this provides 131 bits of message capacity with 124 parity bits.[53]

Reed-Solomon Codes

Reed-Solomon (RS) codes are non-binary BCH codes operating on symbols (groups of bits) rather than individual bits. They excel at correcting burst errors common in memory-based PUFs.[60][57][56]

RS(n, k) parameters work in $GF(2^m)$ (Galois field):

- Common configuration: RS(255, 233) over $GF(2^8)$, correcting up to 11 symbol errors
- Each symbol is 8 bits, so this corrects up to 88 bit errors in bursts[60]

Usage in PUFs: RS codes often provide first-stage correction in concatenated schemes, followed by BCH codes for fine-grained correction.[57][56]

Low-Density Parity-Check (LDPC) Codes

LDPC codes offer better performance than BCH/RS for longer block lengths but with higher decoding complexity. Recent UAV-PUF protocols use quasi-cyclic LDPC (QC-LDPC) with optimized bit-flip decoding to reduce computational overhead while achieving low decoding failure probability.[12][53]

Implementation Strategy for UAV PUFs

Two-stage error correction:[12]

1. **On-chip correction** (UAV side): Lightweight BCH or optimized LDPC for fast initial correction (< 5 ms)
2. **Server-side correction** (if needed): More sophisticated RS or strong LDPC codes for authentication verification

This approach minimizes UAV computational burden while ensuring reliable authentication. Helper data is transmitted during authentication (not stored on UAV), providing resistance to physical capture attacks.[1][12]

PUF-Based Authentication Protocol for UAVs

Protocol Architecture

A typical PUF-based UAV authentication system involves three entities:[2][3][1]

1. **UAVs (Drones)**: Resource-constrained devices with embedded PUF hardware
2. **Ground Station (GS)**: Edge computing nodes that communicate directly with UAVs
3. **Control Center (CC)**: Central authority managing registration and maintaining secure databases

Three-Phase Protocol Structure

Phase 1: Enrollment/Registration

The enrollment phase occurs once in a secure environment before UAV deployment.[61][62][63][64]

Steps:

1. **PUF interrogation**: The Control Center applies a set of challenges $\{C_1, C_2, \dots, C_m\}$ to the UAV's PUF and records responses $\{R_1, R_2, \dots, R_m\}$. [62][64]
2. **Error correction setup**: For each response R_i , encode using BCH/RS to generate helper data:
$$HD_i = \text{Encode}(R_i)$$
Only the helper data is stored in the database, not the raw responses. [58][53]
3. **Identity assignment**: Generate a unique identifier ID_{UAV} and pseudo-identity $PID_{UAV} = H(ID_{UAV} || R_1)$ for anonymity [1][3].
4. **Credential storage**:
 - **Control Center database**:
$$(ID_{UAV}, \{(C_i, HD_i)\}_{i=1}^m, PID_{UAV})$$
[1]
 - **UAV storage**: Only PID_{UAV} (no secrets stored on UAV) [1]

- **Ground Station:** Registration parameters shared by Control Center

Security note: This phase must occur in a secure facility with tamper-proof channels to prevent CRP leakage.[61]

Phase 2: Mutual Authentication

When a UAV needs to authenticate with a Ground Station, the following lightweight protocol executes:[63][11][2][1]

Step 1 - UAV Initiation:

- UAV generates timestamp T_1 and random nonce N_1
- Computes authentication request:

$$M_1 = \text{PID}_{\text{UAV}} \| T_1 \| H(T_1 \| N_1)$$
- Sends M_1 to Ground Station[11][63]

Step 2 - Ground Station Challenge:

- GS verifies timestamp freshness: $|T_{\text{current}} - T_1| < \Delta T$ (e.g., 2 seconds)[11]
- Retrieves challenge C_i from database for PID_{UAV}
- Generates nonce N_2
- Computes: $M_2 = C_i \oplus H(N_1) \| N_2$
- Sends M_2 to UAV[63][11]

Step 3 - UAV Response Generation:

- UAV extracts challenge: $C_i = M_2 \oplus H(N_1)$
- Generates PUF response: $R'_i = \text{PUF}(C_i)$
- Computes masked response: $R_i^* = R'_i \oplus H(N_2)$
- Generates session key: $SK = H(R'_i \| N_1 \| N_2 \| T_1)$
- Computes authentication tag: $\text{Auth}_{\text{UAV}} = H(R_i^* \| SK \| T_1)$
- Sends $M_3 = R_i^* \| \text{Auth}_{\text{UAV}} \| T_2$ to GS[11][63]

Step 4 - Ground Station Verification:

- GS verifies timestamp: $|T_{\text{current}} - T_2| < \Delta T$
- Unmasks response: $R'_i = R_i^* \oplus H(N_2)$
- Corrects errors using helper data: $R_i = \text{Decode}(R'_i, \text{HD}_i)$ [53]
- Recomputes session key: $SK' = H(R_i \| N_1 \| N_2 \| T_1)$

- Verifies: $\text{Auth}'_{\text{UAV}} = H(R_i^* \| SK' \| T_1)$
- If match, authentication succeeds[11][63]

Step 5 - Ground Station Confirmation:

- GS computes confirmation: $\text{Auth}_{\text{GS}} = H(SK \| R_i \| T_2)$
- Sends $M_4 = \text{Auth}_{\text{GS}}$ to UAV
- UAV verifies to complete mutual authentication[63]

Operations used: Only hash functions (SPONGENT-128/160), XOR, and timestamp comparison. No asymmetric cryptography, exponentiation, or pairing operations.[2][63][1]

Phase 3: Session Key Establishment

After successful authentication, both parties independently derive the same session key SK for encrypted communication:

$$SK = H(R_i \| N_1 \| N_2 \| T_1)$$

This key provides perfect forward secrecy because:

- Fresh nonces N_1, N_2 ensure uniqueness per session
- PUF response R_i adds device-specific entropy
- Timestamp prevents replay[3][50]

For long-duration missions, periodic re-authentication (every 5-10 minutes) with fresh CRPs maintains security.[1]

Security Properties Achieved

The protocol provides:[50][3][2][1]

- **Mutual authentication:** Both UAV and GS verify each other's identity
- **Anonymity:** PID_{UAV} changes periodically; ID_{UAV} never transmitted in plaintext[3][1]
- **Untraceability:** Different sessions use different nonces and CRPs, preventing tracking
- **Replay attack resistance:** Timestamps and nonces ensure freshness
- **Man-in-the-middle resistance:** Hashed authentication tags bind all message components

- **Physical capture resistance:** No secrets stored on UAV; PUF responses change if tampered[13][12][1]
- **Forward secrecy:** Compromised session keys don't reveal past/future keys[50][3]
- **Stolen verifier resistance:** Ground stations don't store raw PUF responses[1]

Implementation Guide

Hardware Requirements

UAV Side (Embedded)

Minimum specifications:

- **Microcontroller:** ARM Cortex-M4 or equivalent (e.g., STM32F4, nRF52840)[44]
- **Clock speed:** 64-168 MHz
- **RAM:** 128-256 KB
- **Flash:** 512 KB - 1 MB
- **PUF implementation:** SRAM PUF (reuse existing RAM) or Arbiter PUF FPGA core[7][12]

PUF hardware options:

1. **SRAM-based** (zero additional hardware):[12]
 - Use existing SRAM in microcontroller
 - Read power-up states for initial key generation
 - Challenge selection via memory addressing
2. **FPGA-based strong PUF** (for frequent authentication):[7][28]
 - 64-128 stage Arbiter PUF or 128+ Ring Oscillator PUF
 - Implement on small FPGA (Lattice iCE40, Xilinx Spartan) connected via SPI
 - Area: < 1000 LUTs for RO-PUF, < 500 LUTs for Arbiter PUF[14][8]
3. **Integrated PUF IP cores** (future designs):
 - SRAM PUF IP blocks available from vendors
 - Can be integrated into SoC designs for custom drone controllers

Ground Station Side

Specifications:

- **Hardware:** Raspberry Pi 4B (representative benchmark)[65][1]
- **Processor:** ARM Cortex-A72, 1.5 GHz quad-core
- **RAM:** 4-8 GB
- **Storage:** 32+ GB SD card for CRP database

Recent tests show Raspberry Pi 4B can handle 170-180 UAV authentications per second using PUF protocols.[66]

Control Center

Standard server infrastructure with secure database for CRP storage and management.

Software Stack

Cryptographic Library

Use lightweight crypto library implementing:

- **Hash functions:** SPONGENT-128/160 or SHA-3 Keccak (NIST-approved)[47][44]
- **Error correction:** BCH codec (recommend libbch or custom implementation)[57][53]
- **Random number generation:** Hardware RNG (TRNG) for nonce/challenge generation

Example libraries:

- **Lightweight Crypto** (GitHub: sebastien-riou/lightweight-crypto)
- **WolfSSL Embedded** (includes SHA-3, optimized for ARM)
- **TinyECC** (for future hybrid PUF-ECC schemes if needed)

PUF Interface Software

SRAM PUF reading (pseudocode):

```
function generate_puf_response(challenge):  
    // challenge determines which SRAM addresses to read  
    addresses = derive_addresses(challenge)
```

```

// Power cycle required for SRAM PUF
power_off_sram_region()
delay(10ms) // ensure complete discharge
power_on_sram_region()
delay(1ms) // allow settling

response = []
for addr in addresses:
    response.append(read_sram_bit(addr))

return response

```

Arbiter PUF evaluation (if using FPGA):

```

function evaluate_arbiter_puf(challenge):
    // Configure multiplexer stages with challenge bits
    configure_mux_chain(challenge)

    // Trigger race condition
    start_pulse()

    // Read arbiter decision
    response_bit = read_arbiter_output()

    return response_bit

```

For multi-bit responses, repeat with different challenge variations or use multiple parallel arbiter chains with XOR combination.[8]

Authentication Protocol Implementation

UAV client code (Python/C pseudocode):

```

def authenticate_with_ground_station(puf, pid_uav, gs_address):
    # Step 1: Initiate authentication
    t1 = current_timestamp()

```

```

n1 = generate_random_nonce(128)
m1 = pid_uav || t1 || hash(t1 || n1)
send_to_gs(m1)

# Step 2: Receive challenge
m2 = receive_from_gs()
challenge = m2[:n_bits] ^ hash(n1)
n2 = m2[n_bits:]

# Step 3: Generate PUF response
response = puf.evaluate(challenge)
masked_response = response ^ hash(n2)

# Step 4: Compute session key and auth tag
session_key = hash(response || n1 || n2 || t1)
auth_tag_uav = hash(masked_response || session_key || t1)

t2 = current_timestamp()
m3 = masked_response || auth_tag_uav || t2
send_to_gs(m3)

# Step 5: Verify GS confirmation
m4 = receive_from_gs()
auth_tag_gs = hash(session_key || response || t2)
if m4 == auth_tag_gs:
    return session_key # Authentication successful
else:
    return None # Authentication failed

```

Ground Station server code:

```

def verify_uav_authentication(crp_database):
    # Step 1: Receive authentication request
    m1 = receive_from_uav()
    pid_uav, t1, auth_hash = parse(m1)

    # Verify timestamp freshness

```

```
if abs(current_time() - t1) > 2: # 2 second threshold  
    return "Expired"
```

```
# Step 2: Send challenge
```

```
challenge, helper_data = crp_database.get_crp(pid_uav)  
n2 = generate_random_nonce(128)  
m2 = (challenge ^ hash(n1)) || n2 # n1 from step 1  
send_to_uav(m2)
```

```
# Step 3: Receive response
```

```
m3 = receive_from_uav()  
masked_response, auth_tag_uav, t2 = parse(m3)
```

```
# Verify timestamp
```

```
if abs(current_time() - t2) > 2:  
    return "Expired"
```

```
# Unmask and error-correct response
```

```
response_noisy = masked_response ^ hash(n2)  
response_corrected = bch_decode(response_noisy, helper_data)
```

```
if response_corrected is None:  
    return "Too many errors"
```

```
# Verify authentication tag
```

```
session_key = hash(response_corrected || n1 || n2 || t1)  
auth_tag_expected = hash(masked_response || session_key || t1)
```

```
if auth_tag_expected != auth_tag_uav:  
    return "Authentication failed"
```

```
# Step 4: Send confirmation
```

```
auth_tag_gs = hash(session_key || response_corrected || t2)  
send_to_uav(auth_tag_gs)
```

```
return session_key # Success
```

Performance Optimization

Memory efficiency:

- Store only essential data on UAV: PID_{UAV} (16-32 bytes)
- Database storage per UAV: ID (16 bytes) + 50-100 CRPs $\times$ (challenge + helper data) $\approx$ 10-20 KB/UAV[1]

Computation reduction:

- Precompute hash parameters and BCH syndrome tables
- Use hardware AES acceleration if available for hash function internals
- Batch PUF evaluations during enrollment to amortize setup overhead

Communication optimization:

- Compress messages using lightweight encoding
- Typical message sizes: M1 (80 bytes), M2 (48 bytes), M3 (96 bytes), M4 (32 bytes)
- Total communication overhead: $\sim$ 256 bytes per authentication[2][1]

Testing and Validation

Enrollment Testing

1. Characterize PUF quality:

- Generate 1000+ response samples per challenge at nominal conditions
- Calculate intra-HD (reliability): target $< 3\%$ [21][22]
- Test across temperature range (-20°C to 60°C): verify BER $< 10\%$ [24][21]
- Test across voltage range ($\pm 10\%$ of nominal): verify BER $< 5\%$

2. Uniqueness validation:

- Enroll 5-10 UAVs with same challenge set
- Calculate inter-HD between all pairs: target 45-55%[18][20]
- Check uniformity: each response should have 40-60% ones[20]

3. Error correction tuning:

- Choose BCH code parameters based on worst-case BER
- Verify successful decoding across temperature/voltage range
- Test decoding failure rate: target $< 10^{-6}$ [53]

Protocol Security Testing

Use formal verification tools:

- **ProVerif**: Automated protocol verification tool[67][50][2]
- **AVISPA**: Automated Validation of Internet Security Protocols and Applications[51][50][2]
- **Scyther**: Protocol security verification[68][3]

Test attack resistance:

- **Replay attacks**: Verify timestamp/nonce rejection
- **Man-in-the-middle**: Attempt message modification detection
- **Modeling attacks**: Train ML models on leaked CRPs (if strong PUF), verify resistance
- **Physical attacks**: Test PUF response changes after tampering

Performance Benchmarking

Measure on actual hardware:

- **Authentication latency**: Target < 50 ms end-to-end[34][1]
- **PUF evaluation time**: 5-15 ms for strong PUF[1]
- **Hash computation**: < 1 ms per operation with SPONGENT
- **Error correction**: 5-10 ms for BCH decoding[12]
- **Energy consumption**: < 10 mJ per authentication[41][44]

Comparison: PUF vs. Traditional Methods

PUF vs. PKI (RSA/ECC)

Aspect	PUF-Based	PKI (RSA-2048/ECC-256)
Key storage	None (generated on-demand)[7][10]	Requires secure non-volatile memory
Authentication time	10-50 ms[1][34]	100-500 ms (signature verification)
Energy per auth	3-10 mJ[44][41]	50-200 mJ (public key ops)
Silicon area	1-5K gate equivalents[14][47]	20-50K GE (ECC accelerator)
Physical attack resistance	High (hardware-rooted)[1][12]	Vulnerable to memory extraction
Scalability	Excellent (no certificate management)	Certificate overhead
Standardization	Emerging	Well-established (X.509, TLS)

Verdict: PUF-based methods are 5-10× faster and more energy-efficient for UAV authentication.[3][1]

PUF vs. Pre-Shared Keys (PSK)

Aspect	PUF-Based	Pre-Shared Keys
Key storage	None[7]	Keys stored in NVM (vulnerable)
Key distribution	None (unique per device)[4]	Requires secure channel for setup
Capture attack resistance	High (PUF changes if tampered)[1][13]	Low (key extracted from memory)
Scalability	Good (no key management)	Poor (key management overhead)
Authentication speed	Similar (both use symmetric crypto)	Similar
Uniqueness	Hardware-enforced[4]	Depends on key generation process

Verdict: PUFs provide superior security against physical attacks, critical for UAVs operating in hostile environments.[2][3]

Weak PUF vs. Strong PUF for UAVs

Aspect	Weak PUF (SRAM)	Strong PUF (Arbiter/RO)
CRP space	Limited (10^2 - 10^3) [6][27]	Exponential (10^{18+})[8][16]
Use case	Key generation only[6]	Authentication protocols[3][7]
Hardware cost	Zero (reuse SRAM) [12]	Low-Moderate (custom logic)[8][14]
Evaluation speed	Slow (requires power cycle)[26]	Fast (< 1 ms per CRP)[8]
Reliability	Moderate (5-15% BER)[21][22]	Good (2-5% BER)[32][17]
UAV suitability	Enrollment & key derivation	Repeated authentication[3][7]

Verdict: Use weak PUFs (SRAM) for initial enrollment and session key derivation; use strong PUFs (Arbiter/RO) for frequent authentication in operational missions.[7][3]

Lightweight Hash Function Comparison

Algorithm	Output (bits)	Hardware (GE)	Throughput (Mbps @ 100 kHz)	Energy Efficiency
SPONGENT-128	128	1,060[47]	2.5	Excellent
SPONGENT-160	160	1,329[47]	2.0	Excellent
PHOTON-128	128	1,120[48]	2.8	Excellent
PRESENT-128 (Hash mode)	128	1,886[47]	3.2	Very good
SHA-256	256	10,868[47]	1.5	Poor
SHA-3 (Keccak-256)	256	5,090	8.5	Moderate

Verdict: SPONGENT-128/160 provides the best hardware efficiency for UAV PUF protocols. For systems requiring NIST standardization, SHA-3 (Keccak) offers a compromise between efficiency and compliance.[47][44][1]

Practical Deployment Considerations

CRP Management

Challenge selection strategy:

- Generate cryptographically random challenges during enrollment
- Never reuse CRPs after authentication (mark as consumed in database)[34][11]
- For n expected authentications over UAV lifetime, enroll $2n$ CRPs for safety margin

- Typical mission: 100-500 CRPs sufficient (1-2 authentications per mission, 100-200 missions)[1]

Database security:

- Encrypt CRP database with AES-256
- Use secure database access controls (principle of least privilege)
- Implement audit logging for CRP access
- Consider distributed storage (blockchain) for high-security applications[69]

Environmental Robustness

Temperature compensation:

- Characterize PUF BER across operating temperature range
- Adjust error correction code strength based on expected conditions
- For extreme environments (-40°C to 85°C), increase BCH error correction capacity by 50%[21]

Aging effects:

- PUF responses may drift over months/years of operation
- Schedule re-enrollment every 6-12 months for long-service UAVs
- Implement adaptive error correction with increasing correction capability over time

Integration with Existing UAV Systems

Compatibility with MAVLink protocol:

- Encapsulate PUF authentication messages in MAVLink custom message types
- Authentication occurs before mission data exchange
- Minimal overhead: 256-byte authentication exchange adds <1% to typical mission traffic

Hybrid approaches:

- Use PUF for initial mutual authentication
- Establish TLS session using PUF-derived session key

- Leverage existing secure communication libraries post-authentication

Regulatory and Standardization

Current status:

- No formal PUF authentication standards for UAVs yet
- IEEE P1935 working group on PUF-based security
- NIST lightweight cryptography project includes hash functions suitable for PUF protocols

Recommendations:

- Follow NIST SP 800-90B for entropy source validation
- Use NIST-approved hash functions (SHA-3) where compliance required
- Document PUF characterization methodology for certification

Conclusion

Physical Unclonable Functions provide an ideal hardware security primitive for resource-constrained UAV authentication systems. By exploiting inherent manufacturing variations, PUFs eliminate the need for stored cryptographic keys, significantly reducing vulnerability to physical capture attacks. The combination of PUF-based identity, lightweight hash functions (SPONGENT/SHA-3), XOR operations, and BCH/RS error correction achieves 5-92% computational efficiency gains and 9-68% communication overhead reduction compared to traditional PKI methods.[3][2][1]

For your semester project on fast authentication between UAVs, the recommended implementation approach is:

1. **PUF selection:** Implement Ring Oscillator PUF on a small FPGA (Lattice iCE40) for strong CRP space and reliability
2. **Hash function:** Use SPONGENT-160 for optimal efficiency-security tradeoff
3. **Error correction:** BCH(255, 131, 18) code provides sufficient correction for typical 2-5% BER

4. **Protocol:** Three-phase mutual authentication with XOR-masked responses and timestamp-nonce replay protection
5. **Testing platform:** Raspberry Pi 4B for ground station, STM32F4 or similar for UAV controller

This configuration achieves <50 ms authentication latency with <10 mJ energy consumption, suitable for rapid multi-UAV swarm authentication scenarios.[34][1]

References

All cited sources are embedded as inline citations throughout the document in format.

References

1. [PUF-Based Secure and Efficient Anonymous Authentication Protocol for UAV Towards Cross-Domain Environments](#) - Cross-domain authentication of drones has played an important role in emergency rescue, collaborativ...
2. [A Lightweight and Trustworthy Authentication Protocol for UAV Networks Based on PUF](#) - With the continuous development of drone remote sensing technology, drone networks have garnered wid...
3. [A Probably Secure and Lightweight PUF-Based Authentication Scheme for 6G-Enabled UAV Networks](#) - Unmanned Aerial Vehicles (UAVs) are poised to play a transformative role across civil, commercial, a...
4. [Physical Unclonable Functions \(PUF\)](#) - Discover how Physical Unclonable Functions utilize manufacturing variations for secure hardware auth...
5. [A RING OSCILLATOR BASED PUF IMPLEMENTATION ON ...](#) - Physical Unclonable Functions (PUFs) are circuit primitives that generate chip specific and unique o...
6. [A Study on FPGA Implementation of Physical Unclonable Functions \(PUFs\)](#) - Physical Unclonable Functions (PUFs) are emerging as a critical hardware security primitive, leverag...
7. [Drone-MAP: A Novel Authentication Scheme for Drone-Assisted 5G Networks](#) - Drones, also called Unmanned Aerial Vehicles

- (UAVs) are attracting significant attention in the rese...
8. [Review on Arbiter Physical Unclonable Function and its Implementation in FPGA for IoT Security Applications](#) - Security is one of the most difficult aspects of advancing the Internet of Things (IoT) in our daily...
 9. [A secure arbiter physical unclonable functions \(PUFs\) for device authentication and identification](#) - Recent fourth industrial revolution, industry4.0 results in lot of automation of industrial processe...
 10. [Secure PUF-Based Authentication Systems - PMC](#) - The Internet of Things faces significant security challenges, particularly in device authentication....
 11. [\[PDF\] PUF based Lightweight Authentication and Key Exchange Protocol ...](#) - This work proposes a lightweight authentication and key exchange protocol utilizing the physically u...
 12. [A Low-Complexity Security Scheme for Drone Communication Based on PUF and LDPC](#) - Due to the limited payload and power of drones, the computational overhead, storage overhead and com...
 13. [A Novel and Efficient Authentication Scheme Based on UAV-UAV Environment](#) - Nowadays, unmanned aerial vehicles (UAVs) are used in various fields due to their high maneuverabili...
 14. [Lightweight Strong PUF for Resource-Constrained Devices](#) - Physical Unclonable Functions are security primitives that exploit the variation in integrated circu...
 15. [Secure Key Generation from Challenge-Response Pair ...](#)
 16. 1
 17. [Arbiter PUF: Uniqueness and Reliability Analysis Using Hybrid ...](#)
 18. [Arbiter PUF: Uniqueness and Reliability Analysis Using Hybrid ...](#) - Intra- and inter-hamming distances are employed by Monte Carlo simulations to estimate uniqueness an...
 19. [\[PDF\] Physically Unclonable Functions - CSE](#) - uniqueness where:
 $\triangleright HD(R_i, R_j)$ is Hamming Distance between n-bit signature of chip i and j. $\triangleright$...
 20. [Breaking down PUF Analysis Metrics | Emmet Friel](#) - cyber security blog
 21. [Design of Sequence Dependent PUF Based on Dual-Port SRAM](#) - With the rapid development of IoT technology and artificial intelligence, the real-time and massive ...

22. [A Reliable Current Starved Inverter PUF Based on SRAM Memory Structure](#) - This paper presents a current starved inverter physically unclonable function (PUF) based on static ...
23. [PG-SPAD PUFs: Reconfigurable Physically Unclonable Functions Using Perimeter-Gated Single-Photon Avalanche Diodes](#) - We present a study on physically unclonable functions (PUFs) derived from the dark signal non-uniform...
24. [A Novel Temperature Resilient Mixed Signal PUF based on Current Starved Inverter for Digital Authentication](#) - In modern semiconductor technology, Physically Un-clonable Functions (PUFs) use production variation...
25. [Novel TCAM-based PUF with improved reliability for hardware-entangled security](#) - Physical unclonable functions (PUFs) are the circuits that extract a number of unique chip signature...
26. [\[PDF\] Reliable Physical Unclonable Function based on Asynchronous ...](#) - Different types of PUFs such as Ring-Oscillator PUF, Arbiter. PUF, SRAM-PUF, and Butterfly PUF have ...
27. [\[PDF\] PUF-based Authentication in IoT against Strong Physical Adversary ...](#) - PUFs are categorized as either weak or strong, depending on their ability to produce unique Challenge...
28. [\[PDF\] IMPLEMENTATION OF A 128-BIT ARBITER PUF ON A FPGA](#) - The RO PUF works by taking between two identical ring oscillators for which outputs are two incremental...
29. [Creating and balancing the paths of arbiter-based physically unclonable functions on FPGA](#) - Objectives. The problem of constructing a new structure of paths of physically unclonable function o...
30. [CycPUF: Cyclic Physical Unclonable Function](#) - Physical Unclonable Functions (PUFs) leverage manufacturing process imperfections that cause propagation...
31. [A Novel Attack on Machine-Learning Resistant Physical Unclonable Functions](#) - Many of today's proposed designs for strong physical unclonable functions (PUFs) are constructed from...
32. [Arbiter PUF: Uniqueness and Reliability Analysis Using Hybrid CMOS-Stanford Memristor Model](#) - In an increasingly interconnected world, protecting electronic devices has grown more crucial because...

33. [UNBIAS PUF: A Physical Implementation Bias Agnostic Strong PUF](#) - ...constraints are imposed to achieve symmetric path delays; and systematic variations can severely ...
34. [\[PDF\] Accelerating PUF-based UAV Authentication Protocols Using ...](#)
35. [\[PDF\] A RING OSCILLATOR BASED PUF IMPLEMENTATION ON FPGA](#) - In this work, we first review basic PUF circuit types, including Optical PUFs, Arbiter PUFs, Ring Os...
36. [A dependable and resource-frugal ring oscillator physically unclonable function](#) - ABSTRACT Silicon Physical Unclonable Function (PUF) is a crucial component for hardware security, pr...
37. [Review of Physically Unclonable Functions \(PUFs\)](#)
38. [Design of A Ring Oscillator Based PUF with Enhanced Challenge Response pair and Improved Reliability](#) - Hardware security is of utmost importance nowadays, with the increased number of threats like modell...
39. [Implementation of Ring-Oscillators-Based Physical Unclonable Functions with Independent Bits in the Response](#) - The paper analyzes and proposes some enhancements of Ring-Oscillators-based Physical Unclonable Func...
40. [Attacking RO-PUFs with Enhanced Challenge-Response Pairs](#) - This paper studies the security of Ring Oscillator Physically Unclonable Function (PUF) with Enhance...
41. [S2RAM PUF: An Ultra-low Power Subthreshold SRAM PUF with Zero Bit Error Rate](#) - The reliability of physical unclonable function (PUF) has become the biggest challenge for key gener...
42. [SPSE-AP: A Lightweight Security Authentication Protocol for SRAM PUF Space Expansion](#) - As a lightweight hardware security primitive, SRAM PUF generates keys for IoT authentication but is ...
43. [\[PDF\] Ring Oscillator PUF Design and Results - Iowa State University](#) - This Butterfly PUF is based on unstable cross-coupled circuits and can be seen in Figure 3. It ba- s...
44. [\[PDF\] Design and Evaluation of Lightweight Cryptographic Primitives for ...](#)
45. [\[PDF\] Analysis of Light-Weight Cryptography Algorithms for UAV-Networks](#) - Specifically, each word undergoes a series of XOR operations with itself shifted to the right by spe...

46. [Lightweight Cryptography in UAV systems](#)
47. [spongent: A Lightweight Hash Function](#) *
48. [SPONGENT: The Design Space of Lightweight Cryptographic](#)
49. [See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/221291933](#)
50. [A PUF-Based Secure Authentication and Key Agreement Scheme for the Internet of Drones](#) - ...2023, Sharma et al. proposed a physical unclonable function (PUF)-based authentication and key ag...
51. [A PUF-Based Secure Authentication and Key Agreement Scheme for the Internet of Drones](#) - ...2023, Sharma et al. proposed a physical unclonable function (PUF)-based authentication and key ag...
52. [A Lightweight and Practical Anonymous Authentication Protocol Based on Bit-Self-Test PUF](#) - ...the implementation complexity and execution overhead. The novel HDA exploits the strategy of bit-...
53. [Cryptographic Key Generation from PUF Data Using ...](#)
54. [Helper Data Masking for Physically Unclonable Function- ...](#)
55. [Reducing amount of helper data in silicon physical unclonable functions \(PUFs\) via lossy compression](#)
56. [arXiv:1501.06698v1 \[cs.IT\] 27 Jan 2015](#)
57. [SRAM-PUF Key Extraction Method and Performance Analysis Based on RS and BCH Codes](#) - The Physical Unclonable Function (PUF) is a unique and nonreplicable p...
58. [\[PDF\] HOST PUF-Based Authentication Protocols ECE 525 ECE UNM 1 ...](#)
59. [PUFBasedAuthenticationII](#)
60. [Reed–Solomon error correction](#) - Reed–Solomon codes $RS(n, k)$ are able to detect and correct multiple symbol errors. By adding $t = n - k$...
61. [MITM- and DoS-Resistant PUF Authentication for Industrial WSNs via Sensor-Initiated Registration](#) - Industrial Wireless Sensor Networks (IWSNs) play a critical role in Industry 4.0 environments, enabl...
62. [Designing secure PUF-based authentication protocols for ...](#) - PMC - The PLAKE protocol runs in two phases: a one-time enrollment phase and a device authentication and k...
63. [A PUF-based lightweight authentication and key agreement protocol for smart UAV networks | IET Communications](#) -

AbstractWith the advancement of information technology and the reduction of costs, the application o...

64. [\[PDF\] Secure Authentication Using PUF-A Hardware Based Approach](#) - 1. Device Initialization – The authentication process begins with the initialization of the device, ...
65. [PUF-Based Secure and Efficient Anonymous Authentication Protocol for UAV Towards Cross-Domain Environments](#) - Cross-domain authentication of drones has played an important role in emergency rescue, collaborativ...
66. [An Enhanced Event-Based Dynamic Authentication Protocol Leveraging Arbiter PUF for IoT Devices](#) - We propose a secure and lightweight authentication protocol tailored for resource-constrained Intern...
67. [PUF-Based Lightweight Authentication and Key Agreement Protocol for Secure Internet of Drones in Smart City](#) - With the introduction of the concept of smart cities and the widespread application of drones in are...
68. [A PUF secured lightweight mutual authentication protocol for multi ...](#) - Integrating PUF technology into each UAV ensures that only authenticated UAVs can communicate over t...
69. [BAZAM: A Blockchain-Assisted Zero-Trust Authentication in Multi-UAV Wireless Networks](#) - ...through the principle of ``never trust and always verify''. We introduce a blockchain-assisted ze...